

An Open Source Stimulator for SSVEP-Based BCIs

Jason Boyd
University of Mississippi
1029 Scarlett Dr.
Oxford, MS 38655
1 662 744 2996

jboyd1@olemiss.edu

Yixin Chen
University of Mississippi
207 Weir Hall
University, MS 38677
1 662 915 7438

ychen@cs.olemiss.edu

ABSTRACT

Steady-State Visual Evoked Potentials (SSVEPs) provide researchers with a mechanism for quickly reading users' inputs into a Brain Computer Interface (BCI) without the need for training or surgery. A visual stimulator is necessary to invoke SSVEP responses. This work describes a stimulator that is accurate, portable, and customizable using open source software packages. Two OpenGL toolkits were used in this project for windowing and timing. Originally, the GLUT toolkit was used, but was replaced by the GLFW toolkit that makes use of **the most accurate system time** available for timing.

Categories and Subject Descriptors

H.5.2 [Information Systems]: User Interfaces – input devices and strategies, interaction styles, screen design, user-centered design.

General Terms

Performance, Design, Experimentation, Human Factors

Keywords

SSVEP, brain-computer interface, graphics

1. INTRODUCTION

In this work, an application to generate flashing objects on a computer monitor using open source software packages was created. The result of this work serves as a stimulator for a Brain Computer Interface (BCI) based upon Steady-State Visual Evoked Potentials (SSVEPs). All SSVEP systems must use some type of stimulator in order to evoke the desired potential from the Electroencephalogram (EEG) signal. Thus far, two major types of stimulators exist: flashing Light-Emitting Diodes (LEDs) and oscillating graphics rendered on a computer monitor. The focus of this project is a stimulator of the latter type. The following illustrates the goals and the novelties of this project: a portable stimulator that operates on a Linux platform without the need for custom hardware; an adaptable stimulator

that has the ability to display a number of customizable shapes at customizable frequencies; and an accurate stimulator that operates at an appropriate range of frequencies without significant error or visual artifacts. To date, researchers have created most BCIs on expensive, custom hardware. Implementing an SSVEP stimulator in a Linux environment without expensive hardware is a step toward more generally available BCIs.

1.1 Steady-State Visual Evoked Potentials

An SSVEP is a cortical response to a visual stimulus corresponding to the frequency at which the stimulus is flashing. An SSVEP will generally start appearing when the visual stimulus exhibits a steady frequency over 6 Hz. Although the neurological mechanisms behind an SSVEP are not currently understood, researchers have widely observed that steadily oscillating stimuli evoke these cortical responses in an EEG [6].

SSVEP-based BCIs are desirable for the following three reasons: (1) non-invasive scalp electrodes for reading EEG signals; (2) negligible user training; and (3) high information-transfer rates. A dependent BCI requires some other physiological system for the interface to function. For the most part, researchers consider SSVEP-based BCI systems dependent because they require a user to move their eyes and look at a specific point. However, some have argued that an independent SSVEP-based BCI is possible if the user merely mentally attends the flashing object without visual attention [8-11].

1.2 Components of an SSVEP-Based BCI

Data acquisition, stimulation, and output resolution are three essential components of every SSVEP-based BCI system. Data acquisition generally consists of scalp electrodes attached to a series of converters and filters. After processing, the data acquisition module will usually transform EEG signals to the frequency domain (generally using Fast Fourier Transform (FFT)) for peak frequency detection. Output resolution translates the data into commands; typically, this component is a classifier that activates some command based on the fundamental frequency of the output. For instance, some BCIs will output characters using an on-screen "keyboard" that has each letter flash at a unique frequency, and others will have a remote-controlled car that can be moved and accelerated by glancing at on-screen buttons. The focus of this project is the third component of an SSVEP-based BCI, the stimulator.

1.3 Stimulator Types

For SSVEP-based BCIs, researchers have developed three major types of stimulators [1]. In the following subsections, we briefly review each type.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee.

ACMSE'12, March 29–31, 2012, Tuscaloosa, AL, USA.

Copyright 2012 ACM 978-1-4503-1203-5/12/03...\$10.00.

1.3.1 Light Stimulators (Custom Hardware)

Light Stimulators use various light sources modulated at the desired frequency. LED is the most commonly used light source. However, there are those that use fluorescent and Xe lights as well. The need to build custom hardware and the inability to quickly reproduce and distribute it are the major disadvantages of this type of stimulator.

1.3.2 Single Graphics Stimulators (Monitor)

Stimulators using single graphics take a set of basic shapes (square, rectangle, circle, arrow, etc.) rendered on a computer screen that are cycled on and off at certain frequencies with an unchanging background. It is easy to reuse and distribute this type of stimulator because it exists entirely in software. The ability to update and make changes is one of the distinct advantages of using a software stimulator.

1.3.3 Pattern Reversal Stimulators (Monitor)

Stimulators using pattern reversal are similar to single graphics, but instead of alternating between background and object, these stimulators alternate between two or more patterns. Utilizing computer graphics rendering, these stimulators swap patterns at a specific frequency to elicit an SSVEP response. Generally, these patterns are two-tone symmetrical matrices that are inverted each iteration to create a flashing stimulator. Example patterns are checkerboards and boxes containing lines of alternating hues.

It seems that researchers who use computer screens are almost evenly divided on whether to use single graphics or pattern-reversal stimulators, but there also is not a clear answer on which type of stimulator performs better.

1.4 OpenGL

This project uses OpenGL for all graphics rendering. OpenGL is the most widely used Application Programming Interface (API) for rendering two-dimensional and three-dimensional graphics applications. It is the most stable and best-supported platform for graphics rendering available for Linux [19].

1.4.1 GLUT

The first few iterations of this project use the OpenGL Utility Toolkit (GLUT) for windowing and timing the rendered graphics [20].

1.4.2 GLFW

The final iterations use GL Framework (GLFW) for windowing and timing. GLFW has native support for X Window based Linux systems. It also uses the most precise available system time for its timing functions [21].

2. Related Work

This chapter discusses the work of various researchers in the area of SSVEP-based BCI stimulators[1,6].

2.1 Single Graphics Stimulators

Single graphics stimulators were the most prevalent type of stimulator in the body of work surveyed [2,3,4,5,7,8,9,10,11,12,13]. The stimulators in these papers were 2 to 13 flashing boxes on a computer monitor. Most of them had frequencies ranging from 5 to 25 Hz. However, some stimulators had much higher frequencies. One paper described a stimulator that flashed 9 boxes at frequencies ranging from 27-43 Hz [3]. Another had 2 boxes flashing at 9 and 35 Hz [11].

All of the single graphics stimulators in the literature reviewed used buttons or boxes flashing on the screen as a stimulator. The users would glance at a box (or, in some cases, mentally attend to the box without actually moving their eyes [8,9,10,11]) and that action, if successful, would evoke the desired potential.

2.2 Pattern Reversal Stimulators

Although pattern reversal stimulators were rarer than their single graphics counterparts, this type of stimulator was applied by several research groups [14,15,16,17]. In these works, pattern reversal stimulators stayed within a range of 5 to 25 Hz. Each experiment described a checkerboard pattern on a computer monitor which was visually attended and, if successful, evoked the desired potential.

2.3 Refresh Rates

The refresh rate of a monitor is an important part of developing an SSVEP-based BCI that can enjoy widespread use. The stimulators described in the literature are implemented on screens with refresh rates ranging from 60 Hz to 170 Hz [2,3,5,6,7,9,10,13,16,17].

3. PROJECT DESIGN

3.1 Development Environment

This project was developed in both the Debian (version 5.0) and Ubuntu (version 10.04) Linux distributions. Linux is a free operating system, thus it is a good choice for developing a project intended to be widely available and usable.

Testing for this project took place on a ViewSonic Optiquest V775 CRT monitor with a refresh rate of 75 Hz and a resolution of 1280 x 1024 pixels. The GPU for this system was an ATI Radeon X300 SE with 128 MB memory. A Lutron LX-102 light meter read the flickering object on the screen and was attached to an Agilent 54621D oscilloscope, which used FFT to transform the data from the light meter to the frequency domain.

The programming language used was C, and the graphics rendering was done with the OpenGL API version 7.0.3-7. The version of GLUT used was 2.4.0-6.1, and the version of GLFW used was 2.6-1. All software was compiled with the GNU Compiler Collection (gcc) version 4.3.2.

3.2 Iteration 1 – OpenGL Rectangles

This project inherited a small application written by Joseph Smith of the University of Mississippi that flashed a single rectangle on the screen.

3.2.1 Nine Flashing Rectangles

In order to create a stimulator with more than one flashing rectangle, there must be an overarching frequency at which the application redraws the frame. The rectangles then appear and disappear at a steady rate in harmony with the overarching frequency. The final version of the nine-rectangle stimulator takes the overarching frequency as an argument then calculates the frequencies of the nine flashing boxes as the second through tenth harmonics of the overarching frequency. The reason for using the harmonics was to encourage a higher overarching frequency (generally 60-100 Hz) while ensuring a good range of flicker frequencies.

An array of colors is kept by the application containing the color value of each box (black and white in this instance). All nine of the boxes are rendered in each frame and the color is swapped

for a box every time a counter reached a value that matches the harmonic frequency assigned to it. The counter updates at the overarching frequency.

This stage of the project uses GLUT for windowing and timing.

3.3 Iteration 2 – Static Bitmap Images

For the second iteration of the project, a stack of images is created in a preprocessing step of the application and these images are cycled in order to flicker boxes depicted in the series of bitmaps. Using a static number of images eliminates the need to render multiple objects on the screen; therefore, the stimulator merely cycles images in the appropriate order.

The user has the ability to change the number of images cycled per second, the size of the boxes (square pixels), the number of boxes, and the frequency of each box in the images. The boxes are always squares. The number of boxes being perfect squares makes formatting the window easier.

The overarching frequency idea is retained from the previous iteration, but instead of indicating said frequency, the user can indicate the number of images cycled in one second. This becomes the overarching frequency of the stimulator.

In order to cut down on memory usage, the preprocessing stage creates binary bitmaps that it stores in the form of a two-dimensional unsigned character array where each row of unsigned characters is treated as an array of bits. A set bit indicates a white pixel on the screen, and an unset bit indicates a black pixel (background) on the screen. The stimulator then renders the bitmaps, one per frame, using the `glBitmap()` function of the OpenGL API.

The function to generate the ordering image order takes advantage of the fact that the stack of images is a three-dimensional array of bits. It loops through each image for each box and sets the bits that correspond to the box. The algorithm for calculating whether a box is set for a given image makes the decision based on the calculation of a threshold value. This threshold represents the switch from being set to unset or vice versa. The threshold will be crossed exactly twice the frequency of the given box, but since frequency values are not always harmonic to the overarching frequency, the threshold will not always fall exactly on a frame refresh.

This iteration also uses GLUT for windowing and timing.

3.4 Iteration 3 - OpenGL Textures

The final iteration takes advantage of a feature of modern Graphics Processing Units (GPUs) – the texture cache. One of the main bottlenecks in displaying computer graphics is sending data from main memory to the GPU. Manufacturers created texture caches in order to avoid this bottleneck when displaying images on the faces of three-dimensional objects. Texture caches are small, extremely fast memory modules on the GPU; textures are two-dimensional images stored within the cache. Instead of creating and storing images in memory, this iteration uses texture mapping to flicker boxes all depicted on a single texture stored on the GPU.

Textures were created for flashing one, four, and eight boxes (Figure 1). These textures contain all of the permutations of the boxes. Using these permutations, it is possible to flash the boxes at any specified frequency. Note that Figure 1 does not show the actual size of the textures. They have been enlarged to show

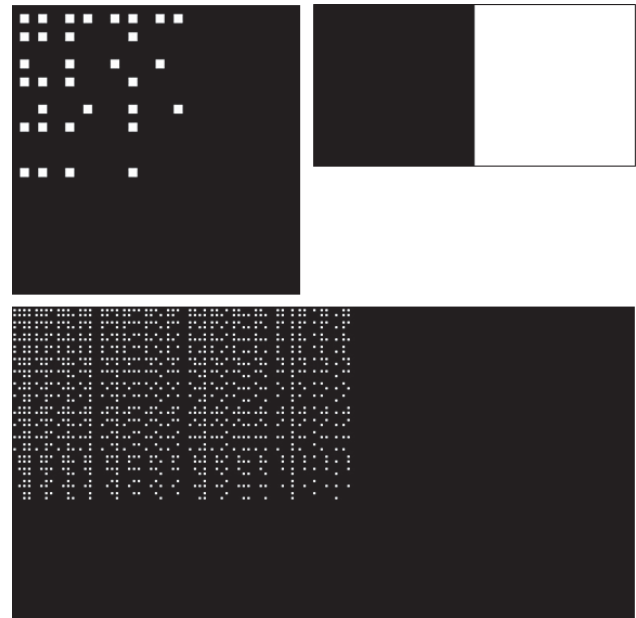


Figure 1. Texture For Flashing One (top right), Four (top left), and Eight Boxes (bottom).

features of the textures. Each box on the four-box and eight-box textures is one pixel in size.

The stimulator runs in a window much larger than the sector of the texture displayed; therefore, the texture sector is stretched to fit onto the OpenGL object. If the stretching is unacceptable, a higher resolution texture can be used.

A slightly modified version of the image selection algorithm described above is used in this iteration. Instead of setting a bit in a binary matrix, the algorithm sets the bit corresponding to the current box in an integer value. The resulting integer represents an index to an array of coordinate values. These coordinates tell the stimulator which sector of the texture to use for the current frame. Each frame a new sector of the texture is displayed.

This iteration initially used GLUT for windowing and timing; however, the resolution of the GLUT timing function seemed to be inadequate. Therefore, the final version of the texture-based stimulator used the GLFW toolkit instead. The GLFW clock functions claim to use the most precise system time available. The experiments conducted for this project support this claim.

To read in the textures, this work used an application that loaded .tga files and packed them in a format usable as OpenGL textures. This application was modified from an original version by Neon Helium Productions [22].

4. RESULTS

This chapter presents the results of the experiments run on the three iterations of this project. For each iteration, collected data is organized into a table (Displayed in Section A, the image appendix). All of the data in this section were gathered using a light meter attached to an oscilloscope. For each trial, the calculated frequency ranged two to thirty (except the last two trials that have an upper bound of 25Hz). The source code of the final, texture-based version of this work is available at <https://sourceforge.net/projects/ssvepstim>.

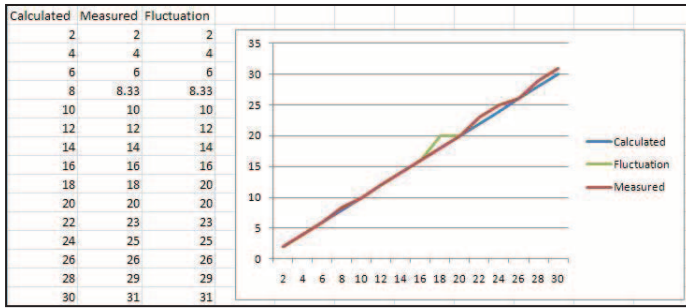


Figure 2. Results From One Rectangle Trial

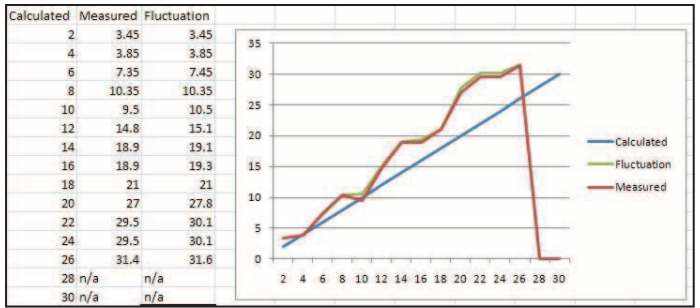


Figure 3. Results From Nine-Rectangle Trial.

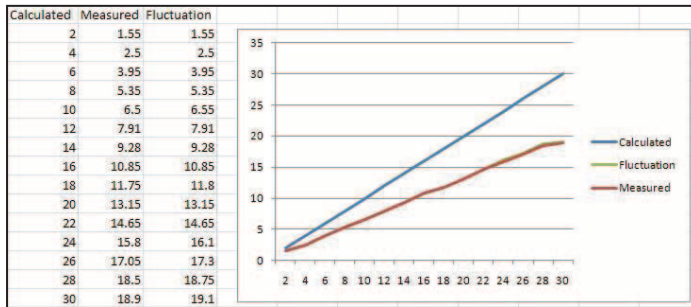


Figure 4. Results From One Box Bitmap Trial.

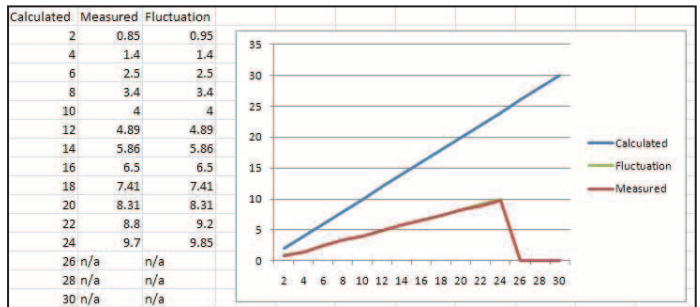


Figure 5. Results From Four-Box Bitmap Trial.

4.1 Results for Iteration 1

There are two sets of results for the first iteration. The first subsection details the results of the stimulator with one flashing OpenGL rectangle, and the second subsection details the results of the stimulator with nine rectangles.

4.1.1 One OpenGL Rectangle

Figure 2 shows the results from the trials with a single flashing OpenGL rectangl. “Calculated” indicates the frequency calculated by the application and “Measured” indicates the measured frequency as presented by the oscilloscope. Fluctuation indicates an upper bound on the measured value; this would only differ from “Measured” when there was no stable peak visible on the oscilloscope.

These results were satisfactory. There was very little variation between the calculated frequency and the measured frequency. Visual artifacts began appearing once the calculated frequency exceeded 14 Hz. These artifacts generally appeared on both sides of the peak in the FFT and were equidistant with the distance from the peak increasing with the frequency. These artifacts were harmonics of some kind, but not harmonics of the peak. Most likely, they are harmonics of a floating frequency that appeared in higher calculated values. One measurement was not stable, but remained close to the actual frequency. This stimulator could be used to elicit SSVEP responses, but it only contains one stimulus, which is not practical.

4.1.2 Nine OpenGL Rectangles

Figure 3 shows the results from the trial with nine flashing OpenGL rectangles. In cases where the signal was noisy and therefore no clear fundamental frequency could be observed, “n/a” was inscribed in measured and fluctuation.

The same artifacts observed in the above trial appeared. This stimulator’s performance is less acceptable than the previous. Inaccuracies in the GLUT timer and the constant re-rendering of OpenGL objects are to blame. The timer is inaccurate for the overarching frequency, which explains how the measured

frequencies could be higher than the calculated. There is much more fluctuation in this trial which means the observed frequencies were not stable. This would severely limit the accuracy of an SSVEP response. Constantly re-rendering the OpenGL objects creates horizontal raster lines scrolling across the rectangles.

4.2 Results for Iteration 2

There are two sets of results for the second iteration. Section 4.2.1 describes a trial with a series of images depicting one box and Section 4.2.2 describes a trial with four boxes.

4.2.1 Images Containing 1 Box

Figure 4 shows the results from the one box bitmap trial. The previous trial revealed that the more boxes present incurred overhead. Therefore, the idea for this iteration was to create a static number of pre-rendered images and the image selection algorithm described in the previous section to create a repeatedly displayed image sequence. The number of boxes would no longer matter since the number of images would never change. However, the OpenGL function used in this iteration was not intended for rapid refreshing; it has prohibiting overhead that caused significant visual artifacts. This, combined with the inaccuracy of the GLUT timer, caused a drastic discrepancy between the measured frequency and the calculated.

There was very little fluctuation in the measured values, but visual artifacts abounded. Flashing horizontal lines of various frequencies pervaded the stimulator, making viewing uncomfortable and causing artifacts to crop up all over the oscilloscope. Although the consistency of the frequencies would make this a viable stimulator, the visual artifacts make this an unlikely choice.

4.2.2 Images Containing 4 Boxes

Figure 5 shows the results from the four-box bitmap trial. The discrepancy between the calculated and the measured frequencies increased in this trial, as does the appearance of artifacts both visually and on the oscilloscope. The reason the

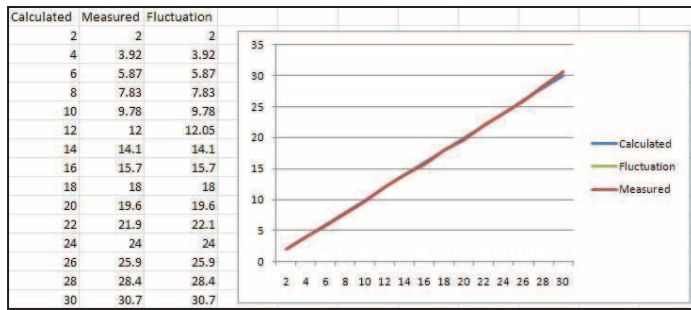


Figure 6. Results From One Box Texture Trial

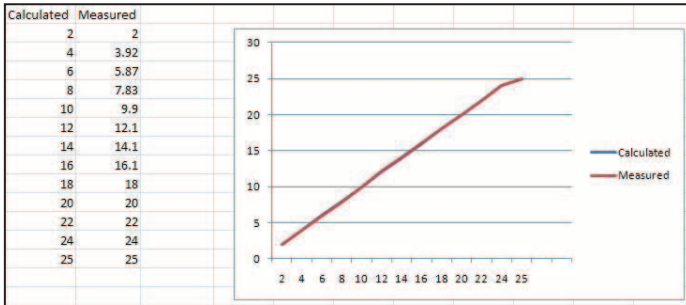


Figure 8. Results From Four-Box Texture Trial With GLFW

artifacts were worse in this trial than in the one box trial was because the sequences of images were more complicated. There was also little fluctuation in this trial, but around 10 Hz, the artifacts overtook the flicker frequency and it was no longer apparent which peak was the strongest. Since higher frequencies cannot be resolved, this stimulator is extremely limited, and the visual artifacts of this stimulator will cause discomfort when viewing.

4.3 Results for Iteration 3

There are four sets of results for the third iteration. Section 4.3.1 describes a trial using a texture depicting one box that is timed using the GLUT timer. Section 4.3.2 describes a trial using a texture depicting four boxes that is timed using the GLUT timer. A trial using a texture depicting four boxes that is timed using the GLFW clock functions is described in Section 4.3.3. Finally, a trial using a texture depicting eight boxes that is timed using the GLFW clock functions is presented in Section 4.3.4.

4.3.1 Texture Depicting 1 Box

Figure 6 shows the results of the texture trial with one box using the GLUT timer. This trial was the starting point for all of the texture trials. There were very few fluctuations in this trial, and the measured frequency stayed very close to the calculated frequency. There were still artifacts resulting from the inconsistencies of the GLUT timer, but they were not greatly imposing and did not create strong artifacts on the oscilloscope. This could operate as a viable single-box stimulator.

4.3.2 Texture Depicting 4 Boxes (GLUT timer)

Figure 7 shows the results of the texture trial with four boxes using the GLUT timer. A higher box count again caused the GLUT timer to produce frequencies higher than expected. However, the curve of the measured frequencies was smooth because the GPU only rendered one object each frame and merely attached the appropriate sector of the texture to that object. The visual artifacts in this trial were worse than in the one box trial because of the difference in complexity. In

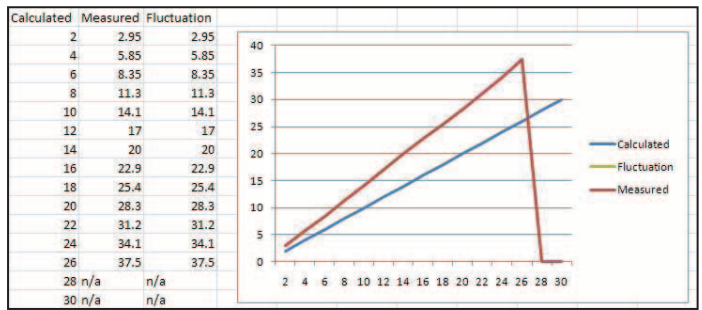


Figure 7. Results From Four-Box Texture Trial With GLUT

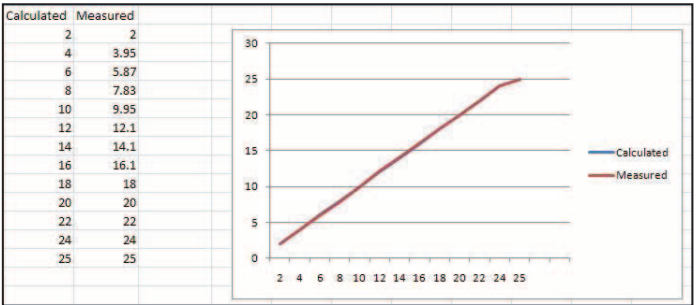


Figure 9. Results From Eight-Box Texture Trial With GLFW

addition, the measured frequency was consistently so much higher than the calculated frequency that it reached the upper bound of allowable flicker frequencies before reaching 30 Hz.

4.3.3 Texture Depicting Four Boxes (GLFW timer)

Figure 8 shows the results of the texture trial with four boxes using GLFW timing functions. With the introduction of timing with GLFW, the visual artifacts all but disappeared and there was no fluctuation of the fundamental frequency. The only artifact left was a slow moving, low frequency raster line. This raster did not appear on a system with a more powerful graphics card than the trial system (ATI Mobility Radeon X300).

The only caveat was that the GLFW buffering functions seemed to have more overhead than their GLUT counterparts did; therefore, the frequency calculation was altered to process half of the sequence per second. The sequence size was hardcoded at 100 for the texture stimulators, which lowers the maximum allowable frequency to 25 Hz. The stimulator in this trial could function as a successful stimulator for an SSVEP-based BCI.

4.3.4 Texture Depicting Four Boxes (GLFW timer)

Figure 9 shows the results of the texture trial with eight boxes using GLFW timing functions. These results reiterated almost exactly the results from the four-box, GLFW texture trial. As long as the texture size does not become cumbersome, this stimulator could flash any number of boxes accurately.

5. DISCUSSION AND CONCLUSIONS

The measured frequency of the texture-based stimulators using the GLFW clock functions matched almost exactly the calculated frequency for every box measured. The lack of artifacts also shows its superiority to the other methods.

Using textures with the GLUT timer produced better results than both other methods for one flashing box, and the results were better for four boxes than the bitmap version. The four-box trial also had better results than the nine-box rectangle trial. Since increasing the number of boxes in the texture version adds no more overhead, these two trials are comparable. Therefore, the

texture-based stimulator, independent of the timer, out-performs the other iterations.

This stimulator can flash shapes of any type at any frequency if the permutations of said shapes are included in the texture. For example, a full alphabet texture stimulator is possible using all of the permutations of the appearance of each letter in the texture. The only real limitation of this method is the texture size. When the texture grows too large, it may bleed over into more than one cache on the GPU.

This work presents results gathered using a light meter. Future research must include results with live subjects using their SSVEP responses to record the accuracy of the stimulator. Several sources in the literature indicated that screen refresh rates (and harmonics) interfered with the signals [1,7,18]; this may limit the ability of the stimulator. These concerns are, however, outside of the scope of this project.

Researching the maximum number of flashing boxes the texture-based stimulator can handle is a useful extension of this research. This is outside of the scope of the project, but will be a good indicator of the texture-based stimulator's viability.

6. REFERENCES

- [1] Zhu, Danhua, Jordi Bieger, Gary Garcia Molina, and Ronald M. Aarts. 2010. A Survey of Stimulation Methods Used in SSVEP-Based BCIs. *Computational Intelligence and Neuroscience*, 2010.
- [2] Bin, Guangyu, Xiaorong Gao, Zheng Yan, Bo Hong, and Shangkai Gao. An online multi-channel SSVEP-based brain-computer interface using a canonical correlation analysis method. *Journal of Neural Engineering*, 2009, volume 6.
- [3] Lin, Zhonglin, Changshui Zhang, Wei Wu, and Xiaorong Gao. Frequency Recognition Based on Canonical Correlation Analysis for SSVEP-Based BCIs. *IEEE Transactions on Biomedical Engineering*, June 2007, volume 54, number 6, 1172-1176.
- [4] Middendorff, Matthew, Grant McMillan, Gloria Calhoun, and Keith S. Jones. Brain-computer Interfaces Based on the Steady-State Visual-Evoked Response. *IEEE Transactions on Rehabilitation Engineering*, June 2000, volume 8, number 2, 211-214.
- [5] Nielson, Kim Dremstrup, Alvaro Fuentes Cabrera, and Omar Feix do Nascimento. EEG Based BCI – Towards a Better Control. *Brain-Computer Interface Research at Aalborg University. IEEE Transactions on Neural Systems and Rehabilitation Engineering*, June 2006, volume 14, number 2, 202-204.
- [6] Wu, Zhenghua, Yongxiu Lai, Yang Xia, Dan Wu, and Dezhong Yao. Stimulator Selection in SSVEP-based BCI. *Medical Engineering & Physics*, 2008, volume 30, 1079-1088.
- [7] Wahnoun, Rémy, Rajiv Saigal, Ying Gu, Nicolas Paquet, Sofie DePauw, Andrew Chen, Saber Sami A. K., and Kim Dremstrup Nielson. A Real-Time Brain-Computer Interface Based on Steady-State Visual Evoked Potentials. *Proceedings of the 7th Annual International Functional Electrical Stimulation Society Conference*, 2002.
- [8] Kelly, S. P., E. Lalor, R. B. Reilly, and J. J. Foxe, Independent Brain Computer Interface Control using Visual Spatial Attention-Dependent Modulations of Parieto-occipital Alpha. *Proceedings of the 2nd International IEEE EMBS Conference on Neural Engineering*, March 2005, 5-8.
- [9] Kelly, Simon P., Edmund C. Lalor, Richard B. Reilly, and John J. Foxe. Visual Spatial Attention Tracking Using High-Density SSVEP Data for Independent Brain-Computer Communication. *IEEE Transactions on Neural Systems and Rehabilitation Engineering*, June 2005, volume 13, number 2,
- 172-178.
- [10] Kelly, Simon P., Edmund C. Lalor, Ciarán Finucane, Gary McDarby, and Richard B. Reilly. Visual Spatial Attention Control in an Independent Brain-Computer Interface. *IEEE Transactions on Biomedical Engineering*, September 2005, volume 52, number 9, 1588-1596.
- [11] Nielsen, Sami S. Communication speed enhancement for visual based Brain Computer Interfaces. *9th Annual Conference of the International FES Society*, September 2004.
- [12] Sugiarto, Indar, Brendan Allison, Axel Gräser. Optimization Strategy for SSVEP-Based BCI in Spelling Program Application. *2009 International Conference on Computer Engineering and Technology*, 2009, 223-226.
- [13] Cheng, Ming, Xiaorong Gao, Shangkai Gao, and Dingfeng Xu. Design and Implementation of a Brain-Computer Interface With High Transfer Rates. *IEEE Transactions on Biomedical Engineering*, October 2002, volume 49, number 10, 1181-1186.
- [14] Martinez, Pablo, Hovagim Bakardjian, and Andrzej Cichocki. Fully Online Multicommand Brain-Computer Interface with Visual Neurofeedback Using SSVEP Paradigm. *Computational Intelligence and Neuroscience*, 2007.
- [15] Trejo, Leonard J., Roman Rosipal, and Bryan Matthews. Brain-Computer Interfaces for 1-D and 2-D Cursor Control: Designs Using Volitional Control of the EEG Spectrum or Steady-State Visual Evoked Potentials. *IEEE Transactions on Neural Systems and Rehabilitation Engineering*, June 2006, volume 14, number 2, 225-229.
- [16] Vasquez, Pablo Martinez, Hovagim Bakardjian, Montserrat Vallverdu, and Andrzej Cichocki. Fast Multi-command SSVEP Brain Machine Interface without Training. *Proceedings of the 18th International Conference on Artificial Neural Networks (ICANN '08)*, September 2008, 300-307.
- [17] Mukesh, T M Srihari, V Jaganathan, and M Ramasubba Reddy. A novel multiple frequency stimulation method for steady state VEP based brain computer interfaces. *Physiological Measurement*, 2006, volume 27, number 1, 61-71.
- [18] Wang, Yijun, Xiaorong Gao, Bo Hong, Chuan Jia, and Shangkai Gao. Brain-Computer Interfaces Based on Visual Evoked Potentials. *IEEE Engineering in Medicine and Biology Magazine*, September/October 2008, 64-71.
- [19] OpenGL Overview. OpenGL. <http://www.opengl.org/about/overview> (Accessed November 15 2010).
- [20] GLUT – The OpenGL Utility Toolkit. OpenGL. <http://www.opengl.org/resources/libraries/glut> (Accessed November 15, 2010).
- [21] GLFW – An OpenGL library. GLFW. <http://www.glfw.org> (Accessed November 15, 2010).
- [22] Neon Helium Productions. Lesson: 33. <http://nehe.gamedev.net/data/lessons/lesson.asp?lesson=33> (Accessed November 15, 2010).